



**ITBA**

Escuela de  
**INNOVACIÓN**

---

EVOLUCIÓN CONTINUA

[innovacion.itba.edu.ar](http://innovacion.itba.edu.ar)

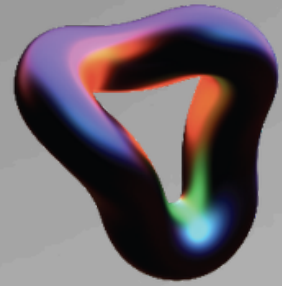
Poner a Grabar





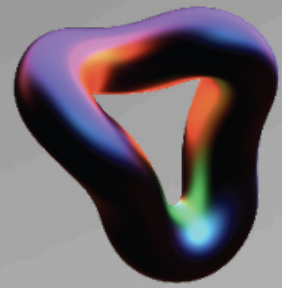
Clase 7  
Sprint 3  
Semana 1  
Node.js

# ¿Qué es Node.js?



Node.js es un entorno de ejecución de JavaScript del  
lado del servidor.

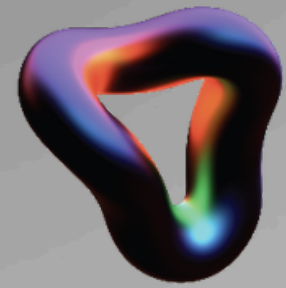
Node.js NO es un nuevo lenguaje de programación.



## ¿Por qué stack MERN?

- JavaScript en Todos Lados: Mismo lenguaje en frontend (React) y backend (Node.js) lo hace increíblemente eficiente. Mismo idioma en ambos lados del proyecto, acelera el desarrollo.
- Rendimiento: Su modelo de operaciones de Entrada/Salida (I/O) asíncrono y no-bloqueante lo hace extremadamente rápido para aplicaciones que manejan muchas conexiones a la vez, como APIs o chats. No se queda esperando a que una tarea termine para empezar la siguiente.
- Ecosistema Gigante (NPM): Node.js viene con npm (Node Package Manager), el gestor de paquetes más grande del mundo. Es una biblioteca infinita de herramientas y librerías gratuitas que podemos instalar y usar para resolver casi cualquier problema, ahorrándonos miles de horas de trabajo.

# Paso a paso

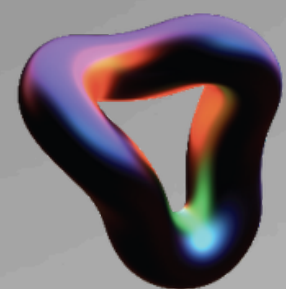


# Paso 1: Instalar Node.js y npm

```
pia@pia-ThinkPad-E470:~$ node -v  
v22.19.0  
pia@pia-ThinkPad-E470:~$ npm -v  
10.9.3
```

## Instalar Node.js:

- Ir a <https://nodejs.org>
- Descargar la versión LTS (Long Term Support)
- Se abre el wizard de instalación. Instala como cualquier otro programa
- Reinicia la PC.



# Paso 2: Crear el proyecto

```
pia@pia-ThinkPad-E470:~/ITBA/monolito$ npm init -y
Wrote to /home/pia/ITBA/monolito/package.json:
```

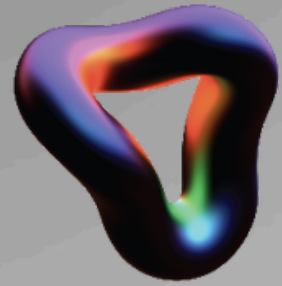
```
{
  "name": "monolito",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
```

`npm init` // inicia la configuración de un proyecto Node.js.  
`-y` // “responder sí automáticamente” a todas las preguntas.

El archivo generado: package.json

```
EXPLORER  ...
└─ MONOLITO
   └─ {} package.json

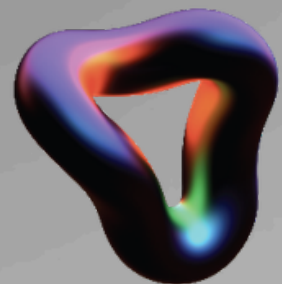
{} package.json x
{} package.json > ...
1  {
2    "name": "monolito",
3    "version": "1.0.0",
4    "description": "",
5    "main": "index.js",
6    "scripts": {
7      "test": "echo \"Error: no test specified\" && exit 1"
8    },
9    "keywords": [],
10   "author": "",
11   "license": "ISC"
12 }
13
```



# Vamos a realizarlo



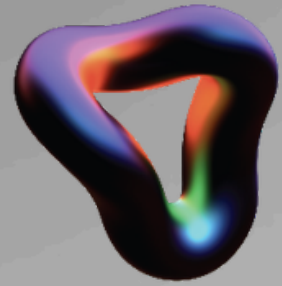
# Módulos Core: Las Herramientas Integradas de Node.js



# ¿Qué son los "módulos core" de Node.js?

Son módulos que vienen preinstalados con Node.js. Algunos ejemplos:

| Módulo | ¿Para qué sirve?                         |
|--------|------------------------------------------|
| fs     | Leer y escribir archivos                 |
| http   | Crear servidores web                     |
| path   | Manejar rutas de archivos                |
| os     | Obtener info del sistema operativo       |
| repl   | Crear entornos interactivos como el REPL |
| crypto | Operaciones criptográficas               |
| events | Manejo de eventos                        |



# ¿Qué herramientas le agrega Node.js a JavaScript?

**1. REPL:** Node.js incluye una herramienta muy útil llamada REPL (Read-Eval-Print-Loop).

Es una consola/terminal interactiva que nos permite escribir y probar código JavaScript al instante, sin necesidad de crear un archivo.

## ¿Cómo se usa?

2

Se abre el entorno REPL de Node.js. Veremos algo como:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
○ pia@pia-ThinkPad-E470:~/ITBA/Clase7_s3_node.js$ node  
Welcome to Node.js v22.19.0.  
Type ".help" for more information.  
>
```

1

Abro una terminal y simplemente escribo

3

Ahora podemos escribir código javascript directamente

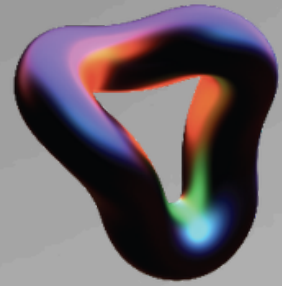
## ¿Qué significa REPL?

REPL es un acrónimo de:

- Read → Lee lo que escribís
- Eval → Evalúa [ejecuta] el código
- Print → Muestra el resultado
- Loop → Vuelve a esperar otra entrada

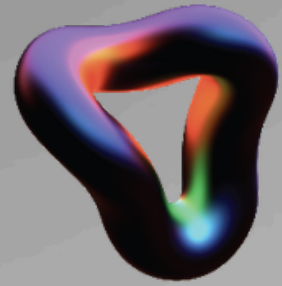
## ¿Para que sirve?

- Probar funciones rápidamente
- Evaluar expresiones
- Testear lógica sin crear archivos
- Explorar objetos y módulos



# Vamos al vscode



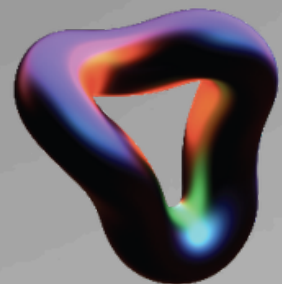


# ¿Qué herramientas le agrega Node.js a JavaScript?

## 2. Sistema de archivos (fs)

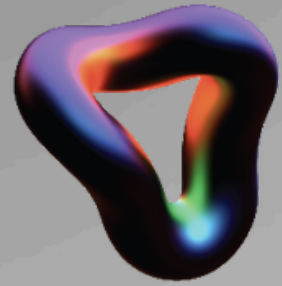
Permite leer, escribir, modificar y eliminar archivos en nuestro disco.

👉 En el navegador no podemos hacer esto directamente por seguridad.



## Algunos métodos del fs

| Método        | Tipo         | ¿Qué hace?                                       | Sincronismo |
|---------------|--------------|--------------------------------------------------|-------------|
| fs.readFile   | Lectura      | Lee el contenido de un archivo                   | Asíncrono   |
| fs.writeFile  | Escritura    | Escribe datos en un archivo (crea o sobrescribe) | Asíncrono   |
| fs.appendFile | Escritura    | Agrega contenido al final de un archivo          | Asíncrono   |
| fs.unlink     | Borrado      | Elimina un archivo                               | Asíncrono   |
| fs.existsSync | Verificación | Verifica si un archivo existe                    | Síncrono    |
| fs.mkdir      | Directorios  | Crea una carpeta                                 | Asíncrono   |
| fs.stat       | Info         | Obtiene información de un archivo                | Asíncrono   |



## 2. Sistema de archivos (fs)

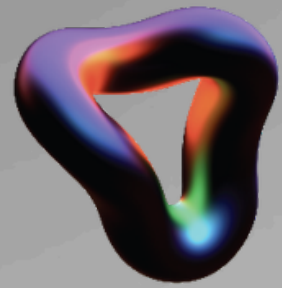
```
const fs = require('fs')  
fs.readFileSync('archivo.txt', 'utf-8')
```

CommonJS Module

```
import { readFileSync } from 'fs'  
readFileSync('archivo.txt', 'utf-8')
```

ES Module

Antes de seguir avanzando,  
veamos que es  
CommonJs y ES Module



# ¿Qué son CommonJS y ES Modules?

Son dos formas distintas de importar y exportar código en JavaScript:

## ◆ CommonJS (CJS)

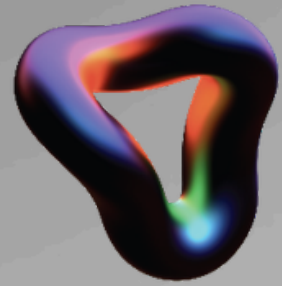
- Sintaxis: `require()` y `module.exports`
- Es el sistema tradicional en Node.js

```
const express = require('express') // importar  
module.exports = app               // exportar
```

## ◆ ES Modules (ESM)

- Sintaxis: `import` y `export`
- Es el sistema moderno usado en navegadores y también soportado por Node.js (desde v12+ con configuración)

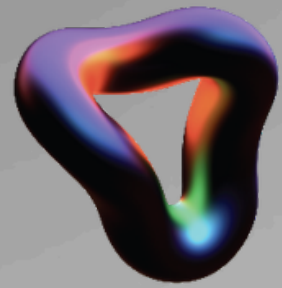
```
import express from 'express' // importar  
export default app           // exportar
```



# ¿Qué son CommonJS y ES Modules?

¿Cuál conviene usar?

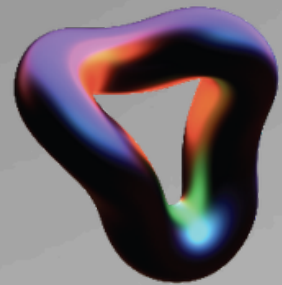
- Si estamos haciendo un **proyecto simple** en Node.js:  
**CommonJS está perfecto.** Es más simple, más compatible con la mayoría de paquetes de npm, y no requiere configuración extra.
- Si querés usar **sintaxis moderna**, **interoperar con frontend**, o **usar import/export**:  
**ES Modules es mejor.** Es el futuro del ecosistema JS.



## Para usar los formatos de ES Module:

Debemos agregar el  
“type”: “module” en mi  
package.json, para avisarle  
a Node.js que voy a usar  
“import” y “export”

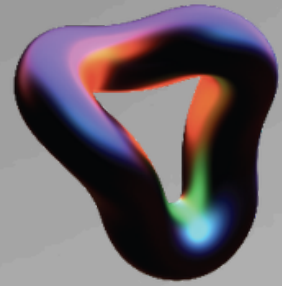
```
{ } package.json X
{ } package.json > ...
1  {
2    "name": "monolito",
3    "type": "module",
4    "version": "1.0.0",
5    "description": "",
6    "main": "index.js",
7    "scripts": {
8      "test": "echo \"Error: no test specified\"",
9    },
10   "keywords": [],
11   "author": "",
12   "license": "ISC",
13   "dependencies": {
14     "express": "^5.1.0"
15   }
16 }
```



# Vamos al vscode



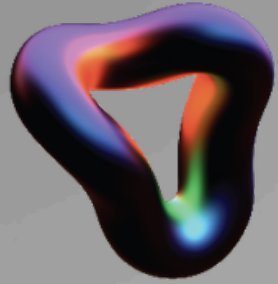
Seguimos viendo las  
herramientas que nos  
provee Node.js



# ¿Qué herramientas le agrega Node.js a JavaScript?

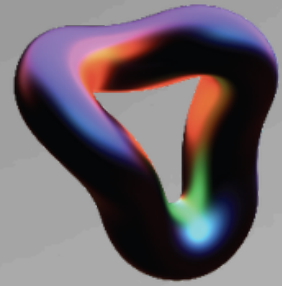
## 2. Path:

Ayuda a trabajar con rutas de archivos y directorios de forma segura y compatible entre sistemas operativos (Windows usa \ y Linux/macOS usa /). Multiplataforma.



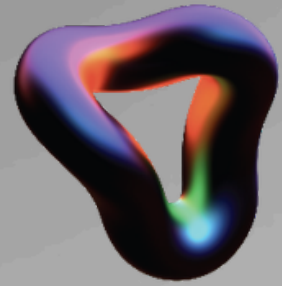
# Algunos Métodos de path

| Método          | ¿Qué hace?                               | Ejemplo                                                            |
|-----------------|------------------------------------------|--------------------------------------------------------------------|
| path.join()     | Une segmentos de ruta                    | path.join('carpeta', 'archivo.txt') → 'carpeta/archivo.txt'        |
| path.resolve()  | Devuelve la ruta absoluta                | path.resolve('archivo.txt') →<br>'/home/user/proyecto/archivo.txt' |
| path.basename() | Devuelve el nombre del archivo           | path.basename('/ruta/archivo.txt') → 'archivo.txt'                 |
| path.dirname()  | Devuelve el directorio padre             | path.dirname('/ruta/archivo.txt') → '/ruta'                        |
| path.extname()  | Devuelve la extensión del archivo        | path.extname('archivo.txt') → '.txt'                               |
| path.parse()    | Devuelve un objeto con partes de la ruta | path.parse('/ruta/archivo.txt') → { root, dir, base, ext, name }   |



# Vamos al vscode





# ¿Qué herramientas le agrega Node.js a JavaScript?

## 3. Red y servidores (http, net)

Permite trabajar con redes y servidores de forma muy directa.

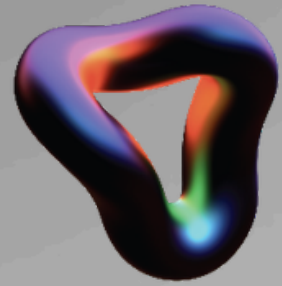
Podés crear servidores web, manejar sockets, hacer peticiones HTTP, etc.

# Algunos módulos para manejo de protocolo de red

| Módulo  | Protocolo / Función | ¿Para qué sirve?                                                              | Ejemplos de uso                                      |
|---------|---------------------|-------------------------------------------------------------------------------|------------------------------------------------------|
| http    | HTTP                | Crear servidores web básicos, manejar solicitudes y respuestas HTTP.          | APIs REST, servidores web simples.                   |
| https   | HTTPS               | Igual que http, pero con soporte para SSL/TLS (conexiones seguras).           | Servidores seguros, login, e-commerce.               |
| net     | TCP                 | Crear servidores y clientes TCP de bajo nivel.                                | Chats en vivo, streaming, protocolos personalizados. |
| tls     | TLS/SSL             | Capa de seguridad sobre TCP, permite cifrado de datos.                        | Servidores seguros personalizados, VPNs.             |
| dgram   | UDP                 | Comunicación sin conexión, más rápida pero sin garantía de entrega.           | Juegos online, VoIP, DNS, streaming en tiempo real.  |
| dns     | DNS                 | Resolver nombres de dominio a direcciones IP y viceversa.                     | Validación de dominios, herramientas de red.         |
| cluster | Multiproceso        | Permite crear múltiples procesos para manejar más conexiones simultáneamente. | Escalabilidad de servidores HTTP o TCP.              |

# Algunos Métodos del módulo http

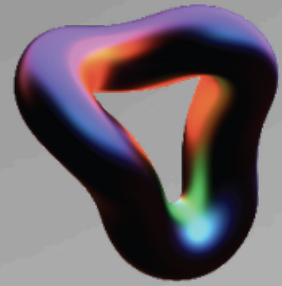
| Método / Propiedad        | ¿Qué hace?                                                                                                              | Tipo      | Ejemplo de uso                                                                                 |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------|-----------|------------------------------------------------------------------------------------------------|
| http.createServer()       | Crea un servidor HTTP que puede manejar solicitudes y enviar respuestas.                                                | Función   | http.createServer((req, res) => { ... })                                                       |
| req.method                | Devuelve el método HTTP de la solicitud (GET, POST, etc.).                                                              | Propiedad | if (req.method === 'POST') { ... }                                                             |
| req.url                   | Devuelve la ruta solicitada por el cliente, normalmente incluyendo la query string, pero no el dominio ni el protocolo. | Propiedad | Cliente solicita:http://localhost:3000/productos?id=5<br>el req.url es "/productos?id=5"       |
| req.headers               | Devuelve los encabezados HTTP enviados por el cliente.                                                                  | Propiedad | console.log(req.headers['user-agent'])                                                         |
| res.writeHead(statusCode) | Especifica el código de estado y los encabezados de la respuesta.                                                       | Método    | res.writeHead(200, { 'Content-Type': 'text/plain' })                                           |
| res.write()               | Escribe datos en el cuerpo de la respuesta.                                                                             | Método    | res.write('Hola mundo')                                                                        |
| res.end()                 | Finaliza la respuesta. Puede incluir datos finales.                                                                     | Método    | res.end('Fin de la respuesta')                                                                 |
| req.on('data', callback)  | Sirve para escuchar eventos('data','end','error') de la petición HTTP                                                   | Evento    | req.on('nombre-del-evento', () => {<br>// código que se ejecuta cuando ocurre el evento<br>}); |



## ¿Para qué sirve todo esto?

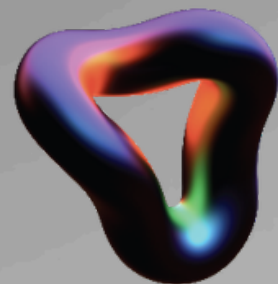
Estos métodos y propiedades nos permiten:

- Crear servidores web básicos sin frameworks.
- Leer solicitudes del cliente (como formularios o JSON).
- Enviar respuestas personalizadas según la URL o el método.
- Manejar headers y datos para construir APIs o páginas dinámicas.



# Vamos al vscode

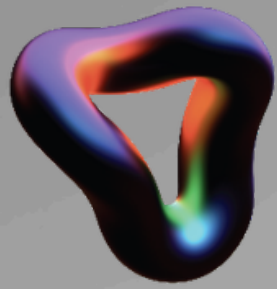




## Status Code HTTP

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status>

| Código | Nombre                | Tipo     | ¿Qué significa?                                                                 |
|--------|-----------------------|----------|---------------------------------------------------------------------------------|
| 200    | OK                    | Éxito    | La solicitud fue exitosa y el servidor devolvió la respuesta esperada.          |
| 400    | Bad Request           | Cliente  | La solicitud tiene una sintaxis incorrecta o está mal formada.                  |
| 401    | Unauthorized          | Cliente  | El cliente no está autenticado. Se requiere login o token.                      |
| 403    | Forbidden             | Cliente  | El servidor entendió la solicitud pero se niega a autorizarla.                  |
| 404    | Not Found             | Cliente  | El recurso solicitado no existe en el servidor.                                 |
| 500    | Internal Server Error | Servidor | Error genérico del servidor. Algo falló internamente.                           |
| 502    | Bad Gateway           | Servidor | El servidor recibió una respuesta inválida desde otro servidor (proxy/gateway). |



# ¿Qué herramientas le agrega Node.js a JavaScript?

## 4. Módulos Personalizados en Node.js

Los módulos personalizados son la forma en que Node.js nos permite organizar y reutilizar código de manera eficiente. Creas un archivo (.js) para agrupar funcionalidades específicas y luego lo exportas para poder usarlo en cualquier otro archivo de tu proyecto. Este enfoque modular ayuda a mantener el código limpio, escalable y fácil de mantener.

### Lo expongo ....

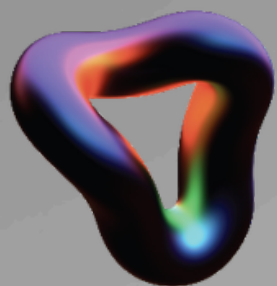
```
function sumar(a,b){  
  return a+b  
}  
function restar(a,b){  
  return a-b  
}  
  
const PI = 3.1415  
  
module.exports = { sumar, restar, PI }
```

```
module.exports = {  
  sumar:sumar,  
  restar:restar,  
  PI:PI  
}
```

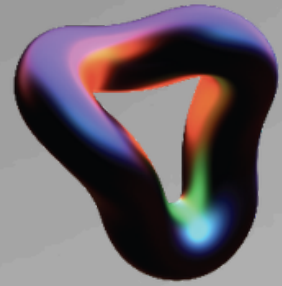
### ... lo consumo

```
const {sumar, restar, PI} = require("./index.js")  
console.log(sumar(1,1))  
console.log(restar(3,1))  
console.log(PI)
```

# NPM y la Gestión de Proyectos en Node.js



¿Qué pasa si necesitamos una funcionalidad más avanzada, como un framework para crear servidores web o una librería para hacer peticiones a otras APIs?

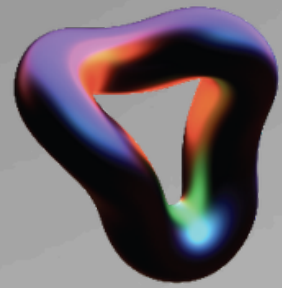


# NPM: El Gigantesco "App Store" para Desarrolladores

NPM (Node Package Manager) es el gestor de paquetes de Node.

## Es dos cosas a la vez:

- Un repositorio online gigantesco (la "tienda") que alberga cientos de miles de paquetes de código abierto (librerías, frameworks, herramientas).
- Una herramienta de línea de comandos (npm) que usamos para instalar y gestionar esos paquetes en nuestros proyectos.



`npm init -y`

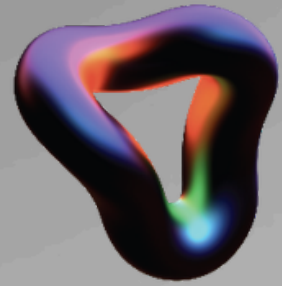
## package.json: El "DNI" de tu Proyecto

Al crear el proyecto con el comando “npm init -y” se genera el package.json.

Cada proyecto de Node.js que se considere serio tiene un archivo package.json en su raíz.

```
EXPLORER  ...  {} package.json x
  v MONOLITO
    {} package.json

  {} package.json > ...
  1  {
  2    "name": "monolito",
  3    "version": "1.0.0",
  4    "description": "",
  5    "main": "index.js",
  6    "scripts": {
  7      "test": "echo \"Error: no test specified\" && exit 1"
  8    },
  9    "keywords": [],
 10    "author": "",
 11    "license": "ISC"
 12  }
 13
```

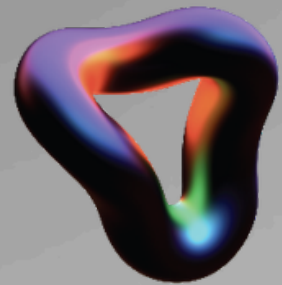


## Instalando paquetes

Cuando instalamos un paquete, este se convierte en una dependencia de nuestro proyecto. Hay dos tipos:

Analogía de la Cocina:

- dependencies (Dependencias de Producción): Son los ingredientes de tu pastel (harina, azúcar, huevos). Tu aplicación final no puede funcionar sin ellas. Ejemplo: Express.
- devDependencies (Dependencias de Desarrollo): Son las herramientas de la cocina que usas para hacer el pastel (batidora, medidores, horno). Son esenciales para el desarrollo, pero no forman parte del pastel final. Ejemplo: Nodemon.



## Instalando paquetes

### Comandos de Instalación

- Para instalar una dependencia de producción:

`npm install express`

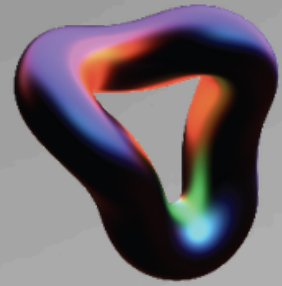
# Forma corta: `npm i express`

- Para instalar una dependencia de desarrollo:

`npm install nodemon --save-dev`

# Forma corta: `npm i nodemon -D`

```
{
  "name": "monolito",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js",
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "express": "^5.1.0"
  },
  "devDependencies": {
    "nodemon": "^3.1.10"
  }
}
```



## En resumen

Node.js convierte a JavaScript en un lenguaje de propósito general, capaz de hacer cosas como:

- Crear servidores
- Leer archivos
- Conectarse a bases de datos
- Manejar redes
- Automatizar tareas

Todo esto fuera del navegador, gracias a esas "herramientas" que le conectaron.

GRACIAS POR ESCUCHARME

/